



SEMESTER 1 EXAMINATIONS 2022/2023

MODULE: CA318 - Advanced Algorithms and A.I. Search

PROGRAMME(S):

CASE BSc in Computer Applications (Sft.Eng.)
ECSAO Study Abroad (Engineering & Computing)

YEAR OF STUDY: 3,O

EXAMINER(S):

Mark Humphrys (Internal) (Ext:8059)

TIME ALLOWED: 2 Hours

INSTRUCTIONS: Detailed instructions inside.

PLEASE DO NOT TURN OVER THIS PAGE UNTIL YOU ARE INSTRUCTED TO DO SO.

The use of programmable or text storing calculators is expressly forbidden.

Please note that where a candidate answers more than the required number of questions, the examiner will mark all questions attempted and then select the highest scoring ones.

There are no additional requirements for this paper.

CA318 - Advanced Algorithms and AI Search - 2023

Instructions:

Answer all 20 multiple-choice questions. (2 marks each.) Total 40 marks.

Answer 2 long questions out of 3. (25 marks each.) Total 50 marks.

Answer the bonus question. (10 marks.)

Remember to hand up the multiple-choice answer sheet!

Multiple choice

1. Playing chess requires what heuristics?
 1. Heuristic evaluation function only.
 2. No heuristics.
 3. Heuristic search, and heuristic evaluation function.
 4. Heuristic search only.
2. An upper bound on the *game-tree complexity* of Xs and Os is:
 1. 9^3
 2. 3^9
 3. 9^9
 4. 3^3
 5. $3!$
 6. $9!$
3. Neural Networks and GAs can both be seen as hill-climbing on an unknown landscape by taking a finite number of samples. We consider E , the error in our solution, and $\frac{\delta E}{\delta w}$ the direction in which to change our solution. Which of the following is the case:

1. For Neural Networks, E is known but $\frac{\delta E}{\delta w}$ is unknown. For GAs, neither E nor $\frac{\delta E}{\delta w}$ are known.
 2. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, E is known and $\frac{\delta E}{\delta w}$ is known.
 3. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, neither E nor $\frac{\delta E}{\delta w}$ are known.
 4. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, E is known but $\frac{\delta E}{\delta w}$ is unknown.
4. Imagine the following in Best-first search. The heuristic evaluation function tells us a state is 4 steps away from the goal. We examine the children of this state and the heuristic evaluation function tells us all of them are 5 steps away from the goal. How can this happen?
1. This cannot happen.
 2. This always happens.
 3. We are in an infinite loop.
 4. The goal has been wrongly defined.
 5. We are going the wrong way.
 6. The heuristic evaluation function is inaccurate.

5. With large state-spaces, it is hard to draw the entire state-space representation of a problem. This is solved by:
 1. Using symmetry to reduce the size of the state-space.
 2. It is not solved. We do not draw the state-space representation of the problem.
 3. Using a heuristic to reduce the size of the state-space.
 4. Doing a truncated search.
 5. Writing a program to automatically draw it.
 6. We draw a partial state-space representation of the problem.
6. A perceptron has 2 inputs, taking values 0 or 1, and 1 output. Its 2 weights are 0 and 1. The output threshold is -0.5. This perceptron implements:
 1. None of the above.
 2. Constant output 0.
 3. AND
 4. XOR
 5. OR
 6. NOT
 7. Constant output 1.
7. Given a line in 2-dimensional space:

$$a x + b y = c$$

Define a perceptron that receives x and y as input and fires if and only if the point is on this side of the line:

$$a x + b y \leq c$$

1. weights: $-a$, $-b$, threshold: c
 2. weights: a , b , threshold: c
 3. weights: a , b , threshold: $-c$
 4. weights: $-a$, $-b$, threshold: $-c$
8. In state-space search, we typically need a function `children(x)` which can generate the child states of state x . We typically use this function to:
 1. Generate a graph of the child states of all states.
 2. Generate a list of the child states of all states.
 3. Generate a list of the child states of a tiny percentage of the states.
 4. Generate a graph of the child states of most of the states.
 5. Generate a graph of the child states of a tiny percentage of the states.
 6. Generate a list of the child states of most of the states.

9. One of these is *known* to be false:

1. Machines have beaten world-class humans at chess.
2. In the future, immortal humans will travel to distant stars.
3. AI is possible.
4. In the future, intelligent machines will exterminate the human race.
5. Machines have beaten world-class humans at tennis.
6. AI (the construction of intelligent machines) is impossible.
7. In the future, humans will live forever on earth in artificial bodies, regularly backing up their brains in order to cheat death.

10. Define a perceptron that receives x and y as input and fires for these points:

$$a x - b y \leq -c$$

1. weights: $-a, -b$, threshold: $-c$
2. weights: a, b , threshold: c
3. weights: $-a, -b$, threshold: c
4. weights: $-a, b$, threshold: $-c$
5. weights: a, b , threshold: $-c$
6. weights: $a, -b$, threshold: c
7. weights: $a, -b$, threshold: $-c$
8. weights: $-a, b$, threshold: c

11. The input to the 7th output node of a neural network is:

$$x_7 = w_{17} y_1 + w_{27} y_2 + w_{37} y_3$$

Then $\partial x_7 / \partial w_{27}$ is:

1. $\sum_j y_j$
2. y_j
3. $\sum_j w_{jk}$
4. w_{jk}
5. w_{27}
6. y_2

12. One of these is *known* to be untrue:
1. In the future, humans will live forever on earth in artificial bodies, regularly backing up their brains in order to cheat death.
 2. AI is possible.
 3. In the future, immortal humans will travel to distant stars.
 4. In the future, intelligent machines will exterminate the human race.
 5. Computers will never be better than humans at chess.
 6. Computers will never be better than humans at football.
 7. AI (the construction of intelligent machines) is impossible.
13. A problem has a *branching factor* of x . To examine all possible states between the current state and n steps into the future requires examining how many states?
1. n
 2. x^x
 3. nx
 4. $x+x^n$
 5. $x + x^2 + x^3 + \dots + x^n$
 6. x^n
 7. x
 8. n^x
14. We are learning from exemplars. We experience 3 exemplars:
 When input is (x_1) output should be (a_1) .
 When input is (x_2) output should be (a_1) .
 When input is (x_1) output should be (a_2) .
 How can this happen?
1. This always happens.
 2. There are missing inputs, or the world is probabilistic.
 3. The machine is badly programmed.
 4. The 3rd exemplar can happen but not the 2nd.
 5. This cannot happen.
 6. The 2nd exemplar can happen but not the 3rd.

15. In trying to predict boxing matches by matching to the nearest exemplar, we have two exemplars:
Opponent height - 186 cm. Opponent weight - 90 kg. Result - I won.
Opponent height - 180 cm. Opponent weight - 95 kg. Result - I lost.

The question is:

Opponent height - 185 cm. Opponent weight - 94 kg. Will I win?

We predict by *Hamming distance* to the nearest exemplar, (a) with height in cm, (b) with height in m. What does it predict?

1. (a) I win, (b) I win.
 2. (a) I lose, (b) I lose.
 3. (a) I win, (b) I lose.
 4. (a) I lose, (b) I win.
16. In machine evolution with asexual reproduction, mutation might break the parent and make it unfit. So therefore:
1. We should mutate the fit, not the unfit.
 2. The original statement is not true.
 3. We should mutate the unfit, not the fit.
 4. It is not something to worry about.
 5. We should only mutate about 1 in a million bits.
 6. We should not mutate.
 7. We should set an appropriate mutation rate for the problem.
 8. We should only mutate at points that will improve the parent.
17. Consider the function $y = f(x) = \zeta(x)$ where ζ is the sigmoid function. f has its (a) highest slope value, and (b) steepest slope (highest absolute value) at:
1. (a) $y = 0$ or 1 , (b) $y = 1/2$
 2. (a) $y = 1/2$, (b) $y = 1/2$
 3. (a) $y = 1/2$, (b) $y = 0$ or 1
 4. (a) $y = 0$ or 1 , (b) $y = 0$ or 1

18. One of these is *not* a property of neural networks:
1. As learning progresses, the *size* of the network remains *fixed*.
 2. Given a network architecture, an Input leads to an Output after a *fixed* number of calculations.
 3. Input is compared to previous Input, and "nearby" Input should produce "nearby" Output.
 4. It can be used to represent a function from real-valued multi-dimensional input to real-valued multi-dimensional output.
 5. It can be used to represent any normal non-linear, continuous, non-chaotic function.
 6. It can be used to represent a function from integer-valued multi-dimensional input to integer-valued multi-dimensional output.
 7. It can be used to represent a function from real-valued single-dimensional input to real-valued single-dimensional output.
 8. It can be used to represent any normal linear, continuous, non-chaotic function.
19. In a perceptron, let inputs be 0 or 1. Let the output be $y = 0$ or 1, and the correct output be C . The Perceptron Learning Rule can be summarised as:
1. If $y=0$ and $C=1$, then reduce threshold and reduce weights.
 2. If $y=1$ and $C=0$, then increase threshold and increase weights.
 3. If $y=0$ and $C=1$, then reduce threshold and increase weights.
 4. If $y=1$ and $C=0$, then reduce threshold and increase weights.
 5. If $y=0$ and $C=1$, then increase threshold and reduce weights.

20. Consider the parallelogram whose boundaries are these 4 lines:

$$y=0, y=3, x=y \text{ and } (x-3)=y$$

A multi-layer neural network architecture to fire only for points *outside* the parallelogram is:

1. 4 hidden units fire for $y \leq 0$, $y \geq 3$, $x \leq y$, and $(x-3) \geq y$, respectively, then feed their outputs into an AND gate.
2. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an AND gate.
3. 4 hidden units fire for $y \leq 0$, $y \geq 3$, $x \leq y$, and $(x-3) \geq y$, respectively, then feed their outputs into an AND gate, then feed the output of that into a NOT gate.
4. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an OR gate, then feed the output of that into a NOT gate.
5. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an OR gate.
6. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an AND gate, then feed the output of that into a NOT gate.

Long questions

Question 1

[Total marks: 25]

- 1(a) Is AI possible? There is no agreed answer to this question, but explain your current thoughts. [7 marks]
- 1(b) "A machine needs a body to understand the world." versus "A machine does not need a body. It can find out some other way what the world is." Discuss. [6 marks]
- 1(c) "A better AI would be one that nailed down the definition of intelligence first, before proceeding." Discuss. [6 marks]
- 1(d) A machine is often expected to learn in 20 hours what a human learns in 20 years. Explain. [6 marks]

Question 2

[Total marks: 25]

- 2(a) Invent a heuristic for either *car buying* or *job hunting*. Explain how it may fail. [5 marks]

2(b) Are the following functions $y = f(x)$ easy or hard to maximise from exemplars? Explain why. [8 marks, 2 each]

i) $f(1.0372) = 80$.
 $f(5.6063) = 100$.
 $f(x) = 0$ for all other x .

ii) $f(1.0372) = 80$.
 $f(1.0373) = 100$.
 $f(x) = 0$ for all other x .

iii) $f(a) = 0$ for some value a .
 $f(x) = 1$ for all other x .

iv) $f(x) = -x^{10}$

2(c) In X's and O's, we considered a heuristic of "Number of winning opportunities". You take the next move with the highest number of winning opportunities. You are done! No search needed. What is the catch? [6 marks]

2(d) Consider breadth-first search: [6 marks, 2 each]

- i) Breadth-first search is a form of *uninformed* search. What does this mean?
- ii) Explain with a diagram how breadth-first search works.
- iii) Discuss the properties, advantages and disadvantages of breadth-first search.

Question 3

[Total marks: 25]

3(a) Consider single-layer networks: [10 marks, 5 each]

- i) What set of weights and thresholds will cause a Perceptron to implement AND?
- ii) Define the 2 perceptrons that fire for the 2 sides of the line:

$$y = a x + b$$

- 3(b) i) In supervised learning with a neural network, we show the network an Input, and then show it the correct Output. The network modifies its weights so that next time it sees the input, it will be *more likely* to generate the correct output. Why such a roundabout method? Why not simply *remember* the correct output and return it *exactly* next time it is asked? [6 marks, 3 each]
- ii) We buy a robot, and find that its sensors measure direction to 3 decimal points, e.g. we get readings like "250.335 degrees". Why is this already a *classification*?
- 3(c) Consider crossing over two parents to produce two offspring. [9 marks, 3 each]
- i) Under what circumstances will the crossover operation *not* produce any new diversity?
- ii) Show how crossover only at gene boundaries produces new diversity but no new genes.
- iii) Show how crossover at random locations produces new genes.

Bonus question (for 10 marks)

*This question is to make you do some reading.
You can prepare this question in advance.*

Tell me about some AI-related algorithm or software that was *not* mentioned on the course.

CA318 - Advanced Algorithms and AI Search

Answer sheet

Black out the square that matches the correct answer.

Exam No: _____

Class: _____

Seat No: _____

Question	1.	2.	3.	4.	5.	6.	7.	8.
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								

Question	1.	2.	3.	4.	5.	6.	7.	8.
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								

